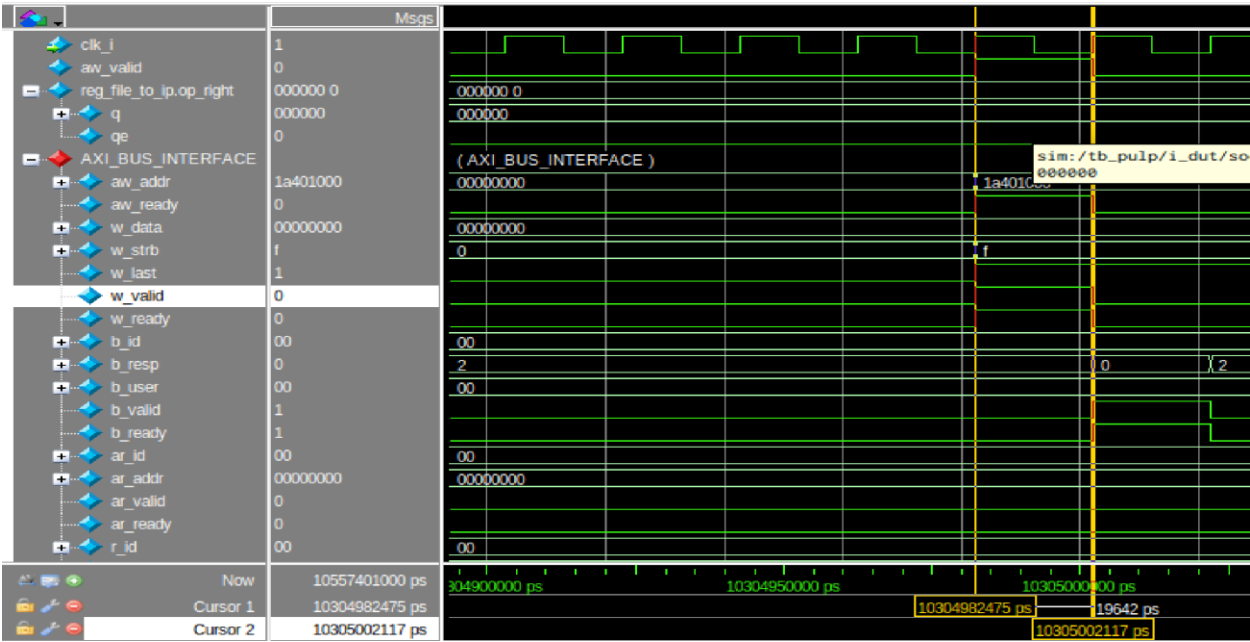


# Latency Profiling

## Clock Period

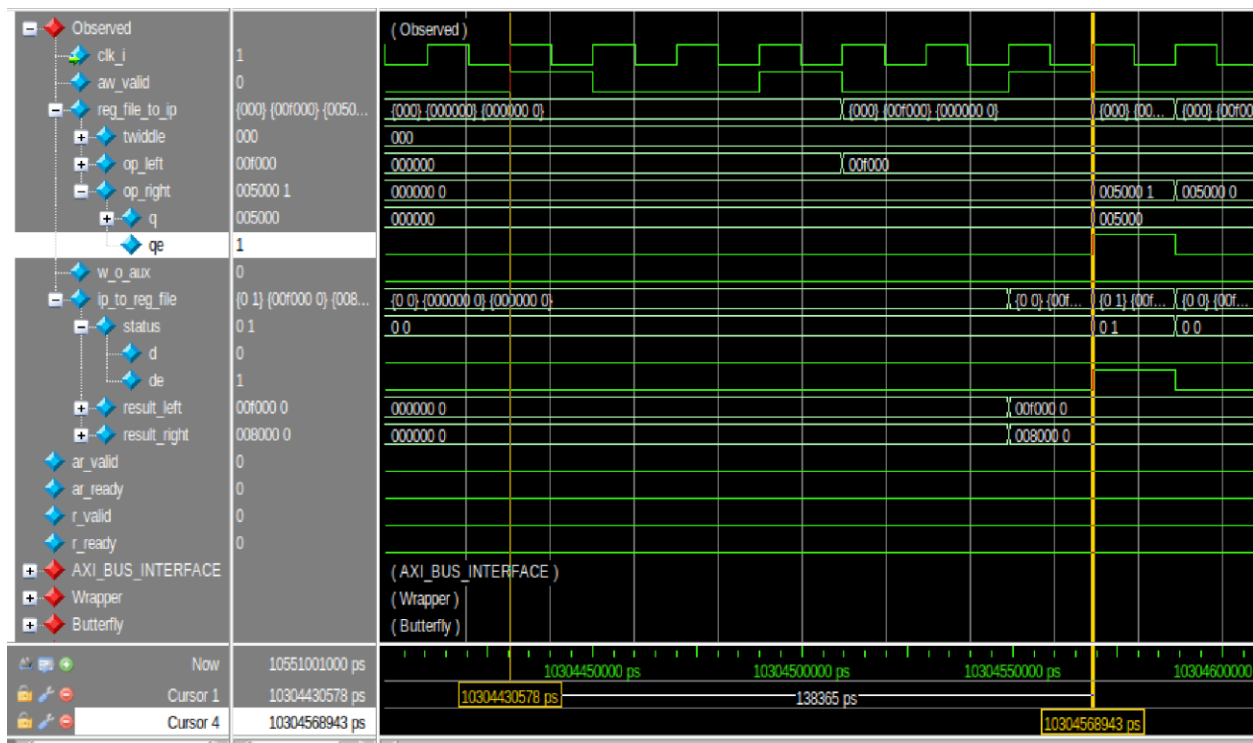


Clock frequency = 50 Mhz, period = 19.669 ns = 20ns

## Data Delivery

How long the RISC-V core and AXI bus take to send *COEF*, *OP\_LEFT*, and *OP\_RIGHT*.

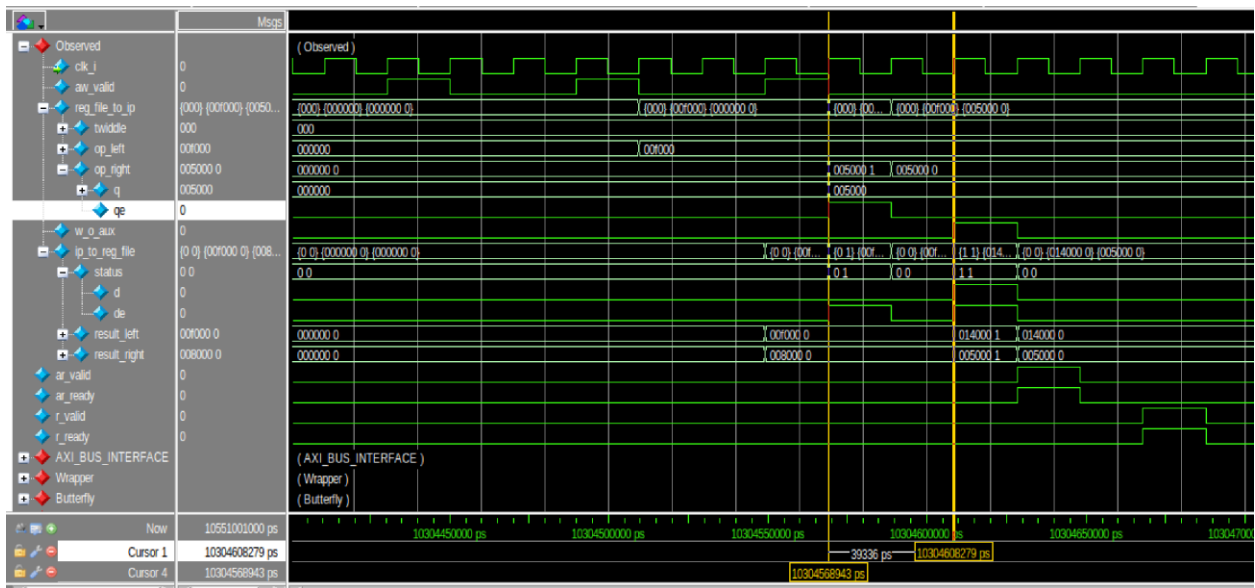
1. **Start Point (Cursor 1):** Find the `AXI_BUS_INTERFACE` signals. Place Cursor 1 on the very first time `aw_valid` goes HIGH (from 0 to 1). This is the start of the first write transaction.
2. **End Point (Cursor 2):** Find your wrapper's trigger signal (reg\_file\_to\_ip.op\_right.qe). Place Cursor 2 on the exact rising edge where `qe` pulses HIGH.
3. **Calculate:** Read the Time Delta, divide by `clk`



$$138365\text{ps} / 19.669\text{ ns} = 7\text{ cycles}$$

*Pure RTL math latency.*

1. **Start Point (Cursor 1):** Leave Cursor 1 exactly where `qe` pulses HIGH.
2. **End Point (Cursor 2):** Find your hardware done flag (`w_o_aux` or `ip_to_reg_file.status.de`). Place Cursor 2 on the exact rising edge where it goes HIGH.
3. **Calculate:** Read the Time Delta, divide by `tclk`

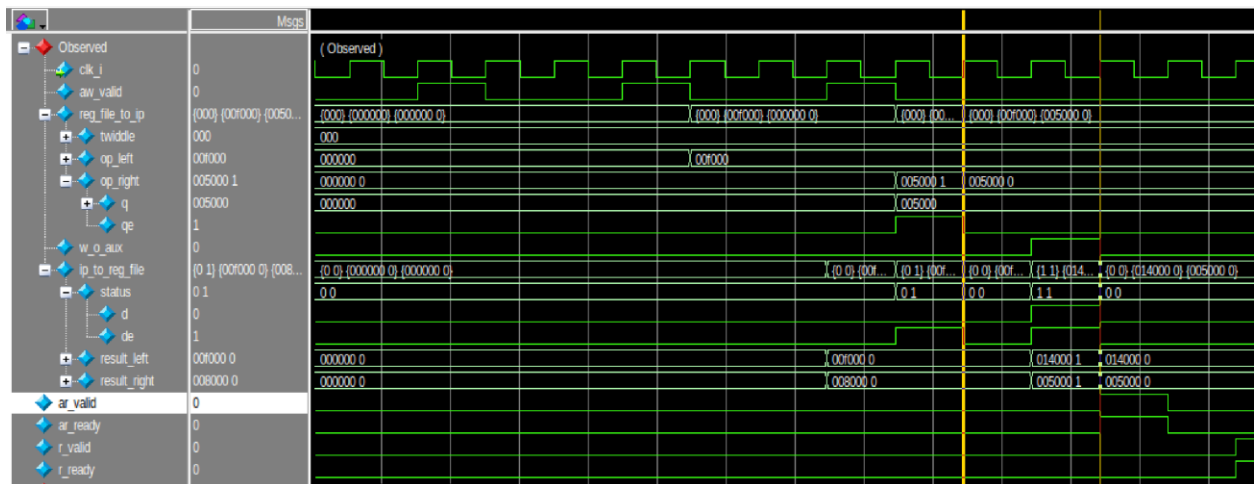


39336 ps / 19.669 ns = 2 cycles (Expected)

## The Software Stalls / Polling Gap

*Pipeline overhead and the 2 `nop` instructions added.*

1. **Start Point (Cursor 1):** Leave Cursor 1 exactly where `w_o_aux` goes HIGH.
2. **End Point (Cursor 2):** Look at the AXI bus signals again. Find the very first time `ar_valid` goes HIGH. This is the moment the CPU wakes up and asks for data. Place Cursor 2 on this rising edge.
3. **Calculate:** Read the Time Delta, divide by `clk`

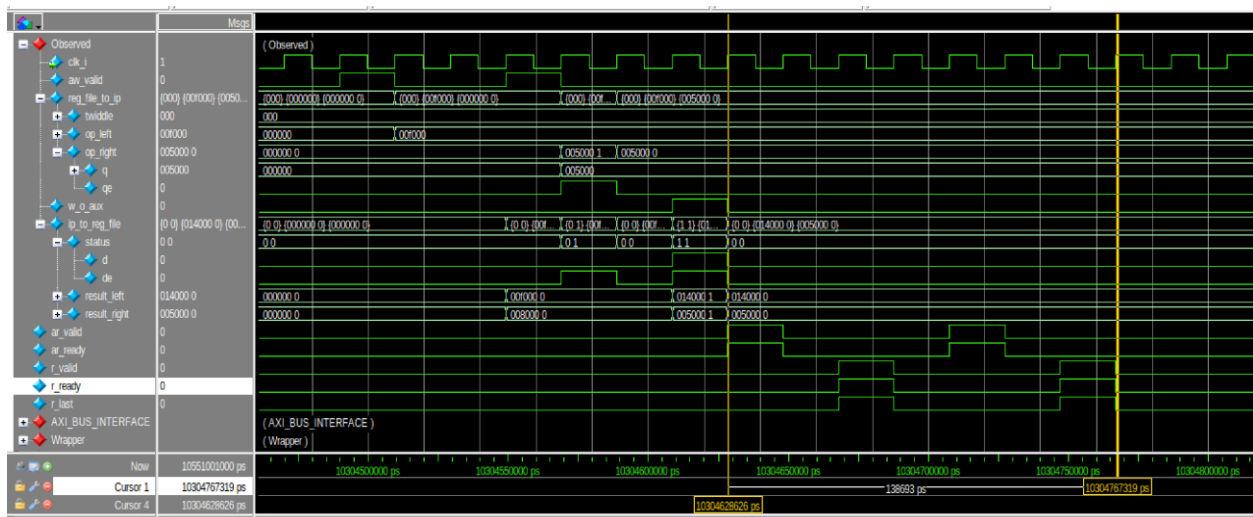


2 cycles (Expected)

## Result Retrieval

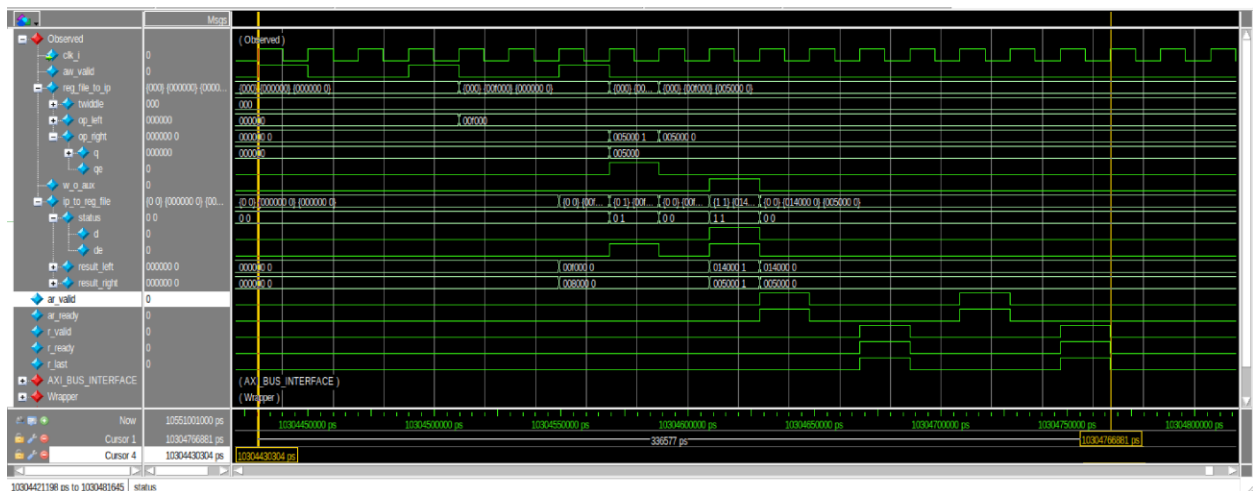
How long it takes to read *STATUS*, *RESULT\_LEFT*, and *RESULT\_RIGHT* over the AXI bus.

1. **Start Point (Cursor 1):** Leave Cursor 1 on that first rising edge of *ar\_valid*.
2. **End Point (Cursor 2):** Look for the final read handshake. Place Cursor 2 on the falling edge of the final *r\_valid* pulse (or the rising edge of the clock immediately following it).
3. **Calculate:** Read the Time Delta, divide by *tclk*



$$138693\text{ps} / 19.669\text{ ns} = 7\text{ cycles}$$

## TOTAL number of cycles



336577ps / 19.669ns = 17 Cycles

Stopwatch cycles perf\_start and perf\_stop

2 cycles (each costs a cycle)

Total Calculated

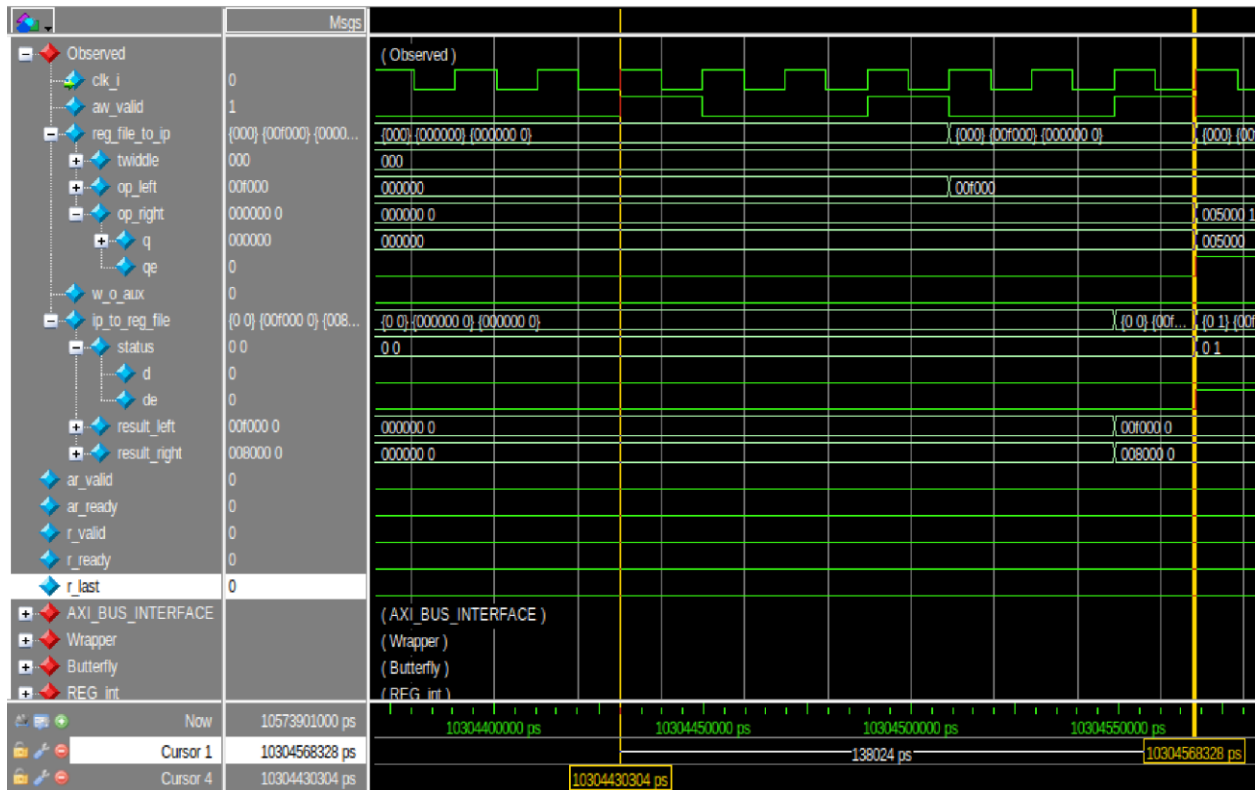
Therefore Total = 19 cycles

```
# [STDOUT-CL31_PE0] -----
# [STDOUT-CL31_PE0] TEST 5
# [STDOUT-CL31_PE0] Twiddle Index : 1024
# [STDOUT-CL31_PE0] LEFT  INPUT   :    50 + j    0
# [STDOUT-CL31_PE0] RIGHT INPUT   :    0 + j    0
# [STDOUT-CL31_PE0] -----
# [HW DEBUG] CYCLE 11502951ns: Auto-Trigger received on OP_RIGHT! i_aux is HIGH
# [HW DEBUG] CYCLE 11502991ns: Pipeline Finished! w_o_aux is HIGH
# [STDOUT-CL31_PE0]
# [STDOUT-CL31_PE0] RESULT 5
# [STDOUT-CL31_PE0] LEFT  OUTPUT  :    50 + j    0
# [STDOUT-CL31_PE0] RIGHT OUTPUT  :    25 + j    0
# [STDOUT-CL31_PE0]
# [STDOUT-CL31_PE0] -> CYCLE LATENCY : 19 clock cycles
# [STDOUT-CL31_PE0] =====
# [STDOUT-CL31_PE0] -----
# [STDOUT-CL31_PE0] TEST 6
# [STDOUT-CL31_PE0] Twiddle Index : 2047
# [STDOUT-CL31_PE0] LEFT  INPUT   :    30 + j    10
# [STDOUT-CL31_PE0] RIGHT INPUT   :    5 + j    2
# [STDOUT-CL31_PE0] -----
# [HW DEBUG] CYCLE 11755251ns: Auto-Trigger received on OP_RIGHT! i_aux is HIGH
# [HW DEBUG] CYCLE 11755291ns: Pipeline Finished! w_o_aux is HIGH
# [STDOUT-CL31_PE0]
# [STDOUT-CL31_PE0] RESULT 6
# [STDOUT-CL31_PE0] LEFT  OUTPUT  :    35 + j    12
# [STDOUT-CL31_PE0] RIGHT OUTPUT  :    12 + j    4
# [STDOUT-CL31_PE0]
# [STDOUT-CL31_PE0] -> CYCLE LATENCY : 19 clock cycles
# [STDOUT-CL31_PE0] =====
```

## Optimization

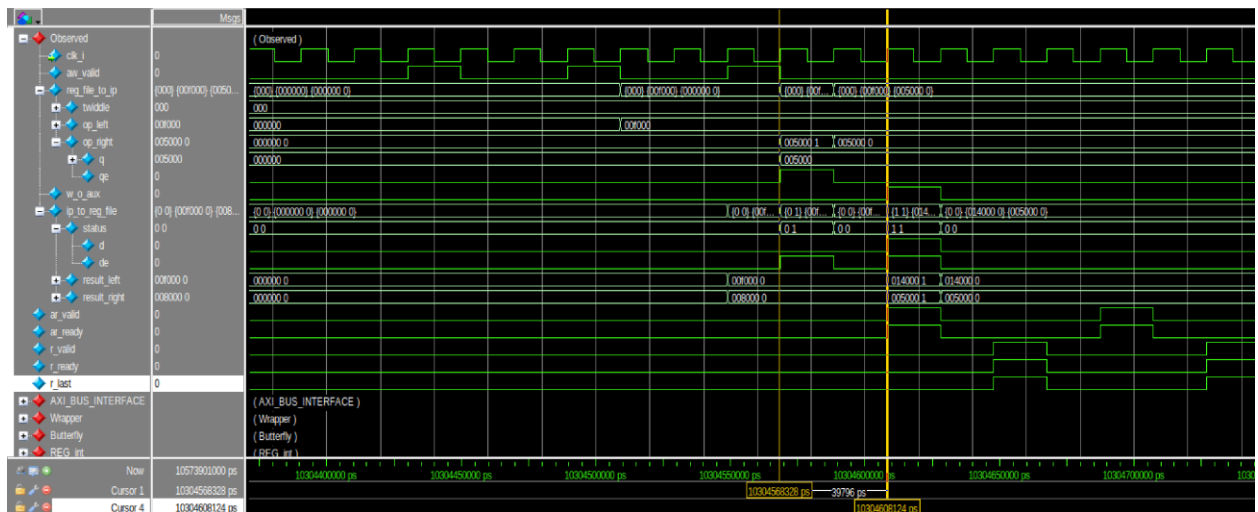
Since Hw is Faster than Sw no need for the polling loop , it will finish before SW asks for the results

## Data Delivery



7 cycles

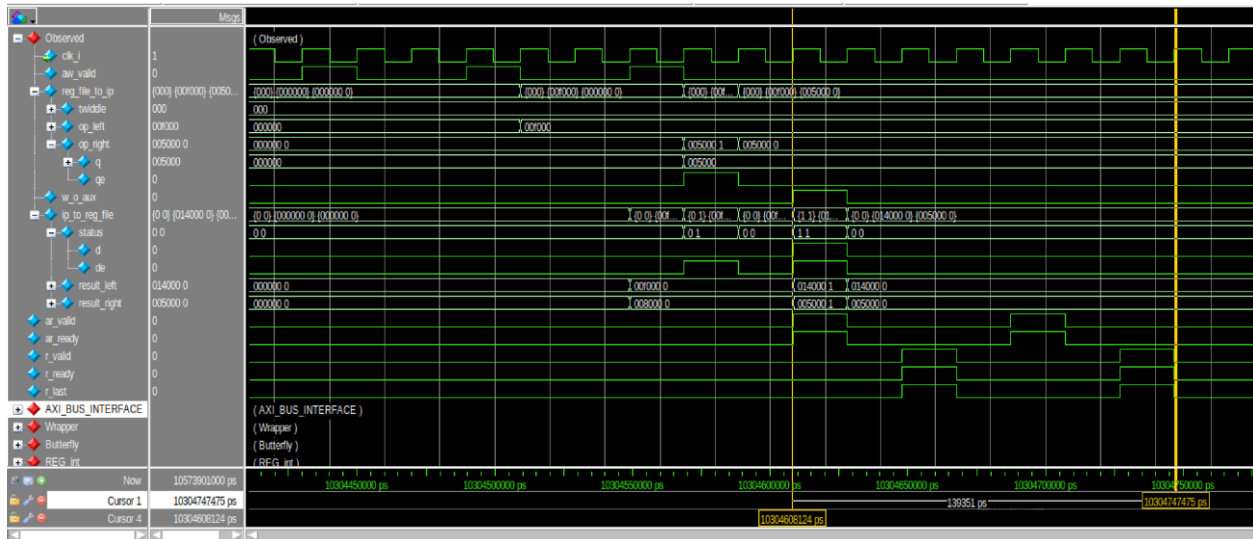
## Hw execution



2 cycles



## Result retrieval



7 Cycles

Total

16 cycles

Calculated Total

18 Cycles

```
# [STDOUT-CL31_PE0] -----
# [STDOUT-CL31_PE0] TEST 6
# [STDOUT-CL31_PE0] Twiddle Index : 2047
# [STDOUT-CL31_PE0] LEFT INPUT : 30 + j 10
# [STDOUT-CL31_PE0] RIGHT INPUT : 5 + j 2
# [STDOUT-CL31_PE0] -----
# [HW DEBUG] CYCLE 11755147ns: Auto-Trigger received on OP_RIGHT! i_aux is HIGH
# [HW DEBUG] CYCLE 11755188ns: Pipeline Finished! w_o_aux is HIGH
# [STDOUT-CL31_PE0]
# [STDOUT-CL31_PE0] RESULT 6
# [STDOUT-CL31_PE0] LEFT OUTPUT : 35 + j 12
# [STDOUT-CL31_PE0] RIGHT OUTPUT : 12 + j 4
# [STDOUT-CL31_PE0]
# [STDOUT-CL31_PE0] -> CYCLE LATENCY : 18 clock cycles
# [STDOUT-CL31_PE0] =====
# [STDOUT-CL31_PE0] -----
# [STDOUT-CL31_PE0] TEST 7
# [STDOUT-CL31_PE0] Twiddle Index : 512
# [STDOUT-CL31_PE0] LEFT INPUT : 40 + j 0
# [STDOUT-CL31_PE0] RIGHT INPUT : 0 + j 0
# [STDOUT-CL31_PE0] -----
# [HW DEBUG] CYCLE 12005407ns: Auto-Trigger received on OP_RIGHT! i_aux is HIGH
# [HW DEBUG] CYCLE 12005447ns: Pipeline Finished! w_o_aux is HIGH
# [STDOUT-CL31_PE0]
# [STDOUT-CL31_PE0] RESULT 7
# [STDOUT-CL31_PE0] LEFT OUTPUT : 40 + j 0
# [STDOUT-CL31_PE0] RIGHT OUTPUT : 20 + j 0
# [STDOUT-CL31_PE0]
# [STDOUT-CL31_PE0] -> CYCLE LATENCY : 18 clock cycles
# [STDOUT-CL31_PE0] =====
```



## Notes:

This proves a fundamental concept in HW/SW Co-design: **You hit the Bus/Software Bottleneck (Amdahl's Law)**. You made the math infinitely faster, but the total time didn't improve because the dominant delay is no longer the multiplier—it is the time it takes the RISC-V core to talk to the AXI bus.

*Pro Tip:* If you can't find it manually, click inside the "sim" window and press Ctrl+F. Search for butterfly3\_top (or the exact module name).

## How to use this .do file next time

Wave.do path:

[/home/icpedia/GP/wave.do](#)

Whenever you close QuestaSim, edit your C code, and run `make run gui=1` again, you will get a blank wave window. Instead of clicking through the hierarchy:

1. Go straight to the Transcript terminal.
2. Type: `do ~/GP/wave.do` and press Enter.
3. *Boom*. All your signals will instantly populate exactly as you have them right now.

## Run Steps to add signals and observe them:

### Generate scripts and build hardware:

#### Bash

```
cd ~/GP
./bender script vsim > sim/compile.tcl
./fix_bugs.sh
cd sim
make build
vopt +acc=npr -o vopt_tb tb_pulp -floatparameters+tb_pulp -work work
```

### **Compile the C code and Launch the GUI:**

Bash

```
cd ~/GP/examples/simple_butterfly_test  
make clean all  
make run gui=1
```

**Add signals or do ~/GP/wave.do**

**In Questasim terminal: run -all**